

## Sustainable Software Engineering - Software Licensing



# Software Licenses

A software license is a **legal contract** between the software creator and the user.

It grants the right to use the software under **specific terms and conditions**:

- It specifies what users can and cannot do with the software.
- It protects the developer's intellectual property.
- It grants the user a license to use it, often in exchange for payment.

These agreements specify **usage rights**, such as device installation limits, modification permissions, and restrictions on copying or redistribution.

# Copyright vs Software License

Copyright is a legal right **automatically granted** to the creator of an original work. It gives the author exclusive rights to:

- reproduce it,
- redistribute it,
- create derivatives.

Copyright exists **by default** once a work is **created** in a tangible form. For example, when you save your Java classes on a disk.

A license is a contract granted by the copyright holder that specifies how others may use the work. It defines

- who can use the work,
- under what conditions (e.g., attribution, share-alike),
- the granted rights (modify, redistribute, commercial use).

A license does not transfer copyright ownership; it only grants **certain rights** under **specific terms**.

# Proprietary vs Open-Source Software

Proprietary software is owned and controlled by an entity that keeps its source code confidential. Open-source software is publicly available and can be freely used, modified, and distributed.

| Feature       | Proprietary Software                                                                | Open-Source Software                                                            |
|---------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Source Code   | Closed and confidential                                                             | Open and accessible                                                             |
| Licensing     | Restrictive license, prohibits modification and redistribution; often requires fees | Permissive licenses allow for modification and redistribution, often at no cost |
| Cost          | Typically requires a purchase or subscription fee                                   | Often free, though paid support or services may be available                    |
| Support       | Provided by the owning company, often with dedicated support systems                | Community-driven support, though paid third-party support is also available     |
| Customization | Limited due to closed source code                                                   | Highly flexible, as users can modify the source code directly                   |
| Development   | Centralized within the company                                                      | Collaborative, community-driven process                                         |
| Examples      | Windows, Adobe Photoshop, Microsoft Office                                          | Linux, React, VS Code                                                           |

# Open-Source Software and CopyLeft

Any derivative work of software must be distributed under the same license terms as the original.

In other words, if you modify or build upon open-source software, you cannot make it proprietary—you must keep it open under the same license.

The original author grants you the right to use, modify, and distribute the software.

In exchange, you agree that any modified version or derivative work will also be licensed under the same copyleft license.

This ensures that the software and its improvements remain free and open for everyone.

# The Free Software Definition

The Free Software Definition is a policy document written by Richard Stallman and published by the Free Software Foundation (FSF).

Freeware and Shareware are not Free Software.

Freeware is copyrighted software that is available at no cost, but unlike free software, its source code is usually closed and cannot be modified.

Shareware is copyrighted software distributed free of charge for a trial period, with full, unrestricted use granted upon purchase.

**4 ESSENTIAL FREEDOMS OF SOFTWARE**

- 0** Run the software whenever you wish, for whatever purpose.
- 1** Study the source code & make modifications to the software.
- 2** Give or sell copies of the software.
- 3** Give or sell copies of your modified versions.

You have the 4 essential freedoms with other useful items that belong to you: clothing, food, simple electrical devices. But most software companies don't want you to have these essential freedoms with software that runs on your various devices, taking away your control over your own devices.

**SWITCH TO FREE SOFTWARE!**

[www.gnu.org](http://www.gnu.org)

# Common Open Source Licenses

| License        | Category                              | Key Differences/Conditions                                                                                                          |
|----------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| MIT            | Permissive                            | Very short and simple; requires only the copyright and license notice be included.                                                  |
| Apache 2.0     | Permissive                            | Similar to MIT but includes an explicit patent grant from contributors and requires a NOTICE file in distributed copies.            |
| BSD (3-Clause) | Permissive                            | Similar to MIT/Apache but includes a clause that prohibits using the author's name for product endorsement without permission.      |
| GNU GPL v3     | Strong Copyleft                       | Requires any modified version or derivative work that is distributed to be released under the same license (or a compatible one).   |
| GNU AGPL v3    | Strong Copyleft (Network Enforcement) | Same copyleft conditions as GPL v3, but also requires license compliance even when the software is run over a network (e.g., SaaS). |
| GNU LGPL v3    | Weak Copyleft                         | Primarily for software libraries; allows linking with proprietary software without forcing the entire work to be open-source.       |

# How to be compliant with licenses when developing software?

Compliance involves adhering to the obligations of each license.  
Not doing so can result in lawsuits or damage to reputation.

The main obligations involve attribution, source code distribution, and patent clauses.

- Attribution: Most licenses require you to credit original authors.
- Source Code Distribution: GPL requires you to provide source code when distributing binaries.
- Patent Clauses: Apache includes a patent grant; GPLv3 explicitly addresses patents.

Always include a LICENSE file in your software projects and add license headers in source files.

# What about mixing licenses?

You should keep a list of third-party components and their licenses.

When combining code from different sources, you must ensure their licenses do not conflict. Incompatible licenses can make your project legally unredistributable.

**Example. Can I combine GPL and MIT code? Yes, you can mix them, but the GPL dominates.**

GPL is a copyleft license that requires the entire combined work to be distributed under GPL. In other words, if you add MIT code to a GPL project, the final project must be GPL. You cannot take GPL code and relicense it as MIT.

The general rule is the following:

- Permissive licenses (MIT, BSD, Apache) can integrate into copyleft projects.
- Copyleft licenses (GPL) impose their terms on the entire combined work.

# How do you choose your license?

You should answer several questions:

Do you want **maximum adoption**? Then, use the MIT or Apache license (permissive).

Do you want to **enforce openness**? Then, use the GPL license (strong copyleft).

Do you want flexibility for **commercial use**? Then, use the Apache or BSD license.

You should also consider community expectations, corporate policies, and patent concerns.

## Risks of ignoring licenses

Companies have been sued for GPL violations (see the BusyBox case study).

Open source communities may blacklist violators. You risk:

- Being excluded from collaborative projects.
- Losing credibility in developer ecosystems.
- Facing public shaming on mailing lists, GitHub issues, and OSS blogs.

Non-compliance can force the removal of infringing code.

# Do you feel lost and demotivated? Let's see how the trenches look

Wintersgill, N., Stalnaker, T., Otten, D., Heymann, L. A., Chaparro, O., Di Penta, M., ... & Poshyvanyk, D. (2025). Developers' Perspectives on Software Licensing: Current Practices, Challenges, and Tools. arXiv preprint arXiv:2510.01096 – <https://arxiv.org/pdf/2510.01096>

arXiv:2510.01096v1 [cs.SE] 1 Oct 2025

## Developers' Perspectives on Software Licensing: Current Practices, Challenges, and Tools

Nathan Wintersgill\*, Trevor Stalnaker\*, Daniel Otten\*, Laura A. Heymann\*, Oscar Chaparro\*,  
Massimiliano Di Penta†, Daniel M. German‡, Denys Poshyvanyk\*  
\*William & Mary (USA), †University of Sannio (Italy), ‡University of Victoria (Canada)  
njwintersgill@wm.edu, twstalnaker@wm.edu, dsotten@wm.edu, lahey@wm.edu, oscarch@wm.edu,  
dipenta@unisannio.it, dmg@uvic.ca, dposhyvanyk@wm.edu

### ABSTRACT

Most modern software products incorporate open-source components, requiring development teams to maintain compliance with each component's licenses. Noncompliance can lead to significant financial, legal, and reputational repercussions. While some organizations may seek advice from legal practitioners to assist with licensing tasks, developers still play a key role in such a process. To this end, it is essential to understand how developers approach license compliance tasks, the challenges they encounter, and the tools that they use. This work studies these aspects of software licensing practices through a study—conducted by a joint team of software engineering and legal researchers—consisting of a survey with 58 software developers and seven follow-up interviews. The study resulted in 15 key findings regarding the current state of practice. We discuss the implications of our findings and offer directions for future research as well as actionable recommendations for licensing tools.

### ACM Reference Format:

Nathan Wintersgill\*, Trevor Stalnaker\*, Daniel Otten\*, Laura A. Heymann\*, Oscar Chaparro\*, Massimiliano Di Penta†, Daniel M. German‡, Denys Poshyvanyk\*. 2025. Developers' Perspectives on Software Licensing: Current Practices, Challenges, and Tools. In . ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

### 1 INTRODUCTION

Software reuse, particularly through open source software (OSS), has become fundamental in the process of software development, allowing software developers to avoid duplicating efforts in areas where previous developers have found suitable solutions. However, the fact that a component is open source does not mean that it can be used without restrictions. Because software is protected by copyright and patent law [5], OSS components are typically released under one or more OSS licenses that govern how a component may be reused, modified, and distributed. Failure to comply with the terms of any applicable license risks liability for infringement, which can result in legal, financial, and reputational consequences [14, 15, 27, 29, 52, 56], both for individual developers and for the organizations for which they work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).  
Conference '17, July 2017, Washington, DC, USA  
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM  
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Ensuring license compliance throughout the development process is, therefore, a critical task. However, this process can be challenging to manage. Software systems can involve hundreds or thousands of OSS components, each under a separate license that must be identified and understood [12]. Furthermore, some OSS licenses have conflicting terms, making the use of components under different licenses problematic. There are hundreds of potentially applicable OSS licenses [7, 8], some of which use legal terminology that is unfamiliar to developers and that does not always have a broadly accepted interpretation [11, 22, 54].

As a result, it may seem appropriate to offload the work of license compliance to legal teams or Open Source Program Offices (OSPOs), but this may delay compliance efforts until near the end of a development cycle [58], leading to lower productivity and wasted effort. Thus, while legal experts play an important role in compliance processes, it is also important for developers to be trained in such tasks.

As the individuals closest to the code, developers bear responsibility for maintaining compliance with the licenses governing that code [58]. However, where previous work has identified that licensing tasks can be challenging [10, 11, 22, 25, 44, 54], it does not identify developers' perceptions of different licensing tasks, and questions remain regarding developers' adoption of licensing tools and the strategies employed when addressing licensing issues.

To fill this knowledge gap, this study surveyed a variety of software developers, including those who may not perform compliance tasks regularly. Our goal was to gain a deeper understanding of the licensing processes and challenges for developers, discover the gaps in the technology developers use for licensing, and work toward organizational and technical solutions.

The main contributions of this paper are the following:

- A novel assessment of how developers not only understand but engage with software licensing tasks, based on a survey of 58 developers with various levels of experience in licensing;
- An in-depth analysis of the processes, roles, and tools that contribute to addressing software licensing issues, including the adoption of available tools and availability of resources in different contexts; and
- Actionable insights into where future licensing tooling development should be directed, including key pain points and opportunities for generative AI.

### 2 BACKGROUND AND RELATED WORK

#### 2.1 Backgrounds on Software licensing

As software is protected by copyright law (among other legal doctrines) in the United States and around the world [5], the owner

## Sustainable Software Engineering - Software Licensing

